

DIPLOMA: INFORMATION TECHNOLOGY

Ubuntu-Connect

AI-Powered Student At-Risk Detection

Project Documentation

Subject:	Business Analysis 3 Module 2 / Business Analysis 3.2
Subject code:	AIBUY3A
Assessment:	Project
Lecturers:	Dr. Sibanda, Mr. Matsela, Mr. Matshego
Group:	Ubuntu-Connect (9 members, listed in the Declaration)
Due date:	02 November 2026

Source code and project board:

https://github.com/SiyabongaBlessingNzotho/Ubuntu-connect

Declaration

We, the members of the Ubuntu-Connect group listed below, declare that this documentation, the accompanying source code and the poster are our own work. Where we used the ideas, data or words of other people, we have referenced them in Section 17. We understand that plagiarism is unethical and illegal and can lead to serious action by the University.

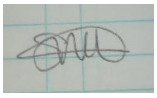
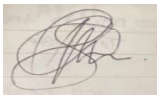
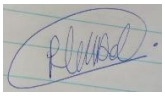
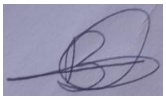
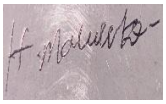
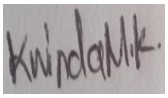
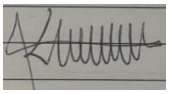
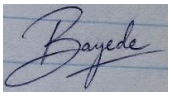
No.	Full name	Student number	Stage and role	Signature	Date
1	Sphamandla Radebe	223762636	Data loading		29/10/2026
2	Siyabonga Nzotho	222156422	Feature engineering (Team Leader)		29/10/2026
3	Pontsho Radebe	219151520	Random Forest baseline		29/10/2026
4	Prudence Mbele	222321598	Deep Learning (Keras)		29/10/2026
5	Khutso Boloka	224555057	NLP sentiment analysis		29/10/2026
6	Humphrey Maluleke	224030272	SMOTE and final model comparison		29/10/2026
7	Esnath Kwinda	224663879	API /predict endpoint		29/10/2026
8	Khanimamba Maluleke	240116194	Chatbot /chat endpoint		29/10/2026
9	Bayede Mthethwa	223673307	Documentation and poster		29/10/2026

Table of Contents

Declaration 2
1. AI Solution and Relevance to the Theme 4
2. Problem Definition 4
    2.1 What is the problem? 4
    2.2 How will solving it with AI benefit the institution and the community? 4
3. Objectives, Success Criteria and Background 5
    3.1 Main objective of the AI solution 5
    3.2 How AI is applied 5
    3.3 Business background 5
    3.4 Business objectives 5
    3.5 Business success criteria 6
4. Requirements, Constraints and Risks 6
    4.1 Requirements 6
    4.2 Constraints 7
    4.3 Risks and mitigation 7
5. Tools and Techniques (initial assessment) 8
6. Machine Learning Approach 9
7. Data 10
    7.1 Source 10
    7.2 Forms of data and examples 10
    7.3 Features used by the deployed model 11
8. Time Series Analysis 11
9. Natural Language Processing 12
10. Deep Learning 12
11. Model Evaluation and Solution Techniques 13
    11.1 How the model is evaluated 13
    11.2 Results (At-Risk class, 209-student test set) 13
    11.3 Techniques used to improve accuracy 14
    11.4 Risk levels on the test set (deployed model) 15
    11.5 Model chosen for deployment 15
12. Other Feature: Chatbot (/chat endpoint) 15
13. Practical Solution 16
    13.1 Real examples from the running API 16
14. Use of AI and Ethical Disclosure 23
15. Project Management and Team 24
16. Conclusion 24
17. References 24
Appendix A: Grammarly Report 26

1. AI Solution and Relevance to the Theme

Ubuntu-Connect is an AI solution for the education industry that gives academic staff an early warning about students who are likely to fail, together with a plain-language reason for every warning. The theme asks for an AI solution that benefits a company in an industry sector of our choice and brings the fourth industrial revolution to people on the ground. Education is one of the industries named in the brief, and it is where early action changes lives: institutions already collect attendance, marks and assignment records, but they rarely analyse them together while there is still time to help. Ubuntu-Connect uses Python machine learning (Random Forest and a Keras neural network), time-series features built from consecutive grading periods, natural language processing of teacher notes, and a FastAPI service with a rule-based chatbot. It turns data that institutions already hold into a prompt for a conversation with the student, using only open-source tools, which makes it affordable for under-resourced institutions. The name reflects the idea of Ubuntu: a student succeeds because the people around them notice and support them.

2. Problem Definition

2.1 What is the problem?

Academic staff usually discover that a student is struggling only after formal results confirm failure, normally at the end of a term. By then little time remains to help. Institutions already collect attendance registers, continuous assessment marks and assignment records, but these are rarely analysed together, so early warning signs such as falling marks, rising absences and previous failures go unnoticed. The result is avoidable failure and dropout, wasted fees and teaching effort, and lost opportunities for students and their families, especially in developing countries where every failed course is costly.

2.2 How will solving it with AI benefit the institution and the community?

A machine learning model reads data the institution already holds and scores each student’s risk before the final exam, with a plain-language reason. Staff can then target tutoring and conversations at the students who need them most. This helps the institution raise pass rates and use limited support staff wisely, and it helps the

surrounding community because more students complete their studies and move into work.

3. Objectives, Success Criteria and Background

3.1 Main objective of the AI solution

The main objective of Ubuntu-Connect is to warn academic staff, early and with a clear reason, about students who are likely to fail, so that staff can support those students while intervention can still change the outcome.

3.2 How AI is applied

AI is applied in four steps. First, supervised machine learning (a Random Forest and a Keras neural network) learns from past students' marks, attendance, previous failures and family background to predict whether a student will fail (a final grade below 10 out of 20). Second, time-series features built from the grading periods and sentiment scores from teacher notes (natural language processing) add the trend in marks and the tone of staff comments. Third, a FastAPI service turns each prediction into a risk level (High, Medium or Low) with a plain-language reason. Fourth, a rule-based chatbot turns the result into practical intervention tips for staff.

3.3 Business background

Student failure and dropout are a cost to the institution, the student and the community. Banking already scores credit risk and healthcare already scores patient risk to act early; Ubuntu-Connect applies the same approach to education. The current process is reactive: staff review results at the end of a term. The proposed process is proactive: staff receive a weekly risk list with reasons and act while intervention can still change the outcome.

3.4 Business objectives

- Identify students at academic risk earlier than the current end-of-term review allows.
- Give academic staff a clear, explainable reason behind every risk flag, not just a raw score.

- Support a measurable, longer-term reduction in avoidable failure and dropout through earlier intervention.

3.5 Business success criteria

Table 1 lists the three success criteria and shows how each one was measured and whether it has been met.

Table 1: Business success criteria and their status

Success criterion	How it is measured	Status in this project
At least 70% precision on the High Risk class	Precision of students flagged High on the 209-student test set	Met: 0.76 (35 of 46 High flags were truly at-risk)
100% of Medium and High flags carry at least one machine-generated reason	Run every student through the /predict endpoint and count flags without a reason	Met: checked on all 1,044 students, which gave 229 High and 33 Medium flags and none without a reason. Low answers have no reason by design
Measurable reduction in the fail rate of flagged students who received intervention	Compare fail rates before and after intervention in a live pilot	Cannot be measured with the historical dataset; planned for a pilot

Criteria 1 and 2 are met on the historical data. Criterion 3 needs real intervention records, so it is reported as planned work and not as a result.

4. Requirements, Constraints and Risks

4.1 Requirements

Functional requirements

- Load and merge the student datasets (Mathematics and Portuguese records).
- Engineer time-series features from the grading periods and add text features from teacher notes.
- Train and compare several models on the same data split.
- Serve a risk score, a risk level (High, Medium or Low) and a reason for a single student through an API (/predict).

- Give staff plain-English intervention tips through a chatbot endpoint (/chat).

Non-functional requirements

- Explainability: every Medium or High flag has a reason.
- Reproducibility: fixed split (80/20, stratified, random_state=42) and saved models.
- Low cost: open-source tools only.
- Usability: staff can use the interactive API page without understanding the model.
- Safety: the system advises staff and never makes decisions about a student by itself.

4.2 Constraints

- No real, consented student data is used; the data comes from the public Student Performance dataset (Cortez and Silva, 2008), downloaded from Kaggle.
- The teacher notes are synthetic because the dataset has no free-text field.
- A 12-week development timeframe and no budget for paid services.
- The test set contains only 46 at-risk students, so small differences between models must be treated with caution.

4.3 Risks and mitigation

Table 2 lists the main risks and what the project does about each one.

Table 2: Risks and mitigation

Risk	Mitigation
Class imbalance (few at-risk students)	class_weight='balanced' and SMOTE on the training data only
False alarms (the model flags students who would have passed)	Three risk levels; reasons shown with every flag; staff confirm with the student before acting
Missed at-risk students	Recall is treated as the priority metric

Risk	Mitigation
Automation bias (staff trusting the score blindly)	Every reply ends with a disclaimer that the score is a prompt to talk, not a verdict
Model bias	Sex is not used as a feature. Some demographic features (age, address, family size, parents' cohabitation status) are used and need a fairness review before real use
Privacy	A real deployment must follow the Protection of Personal Information Act (POPIA) and use consented data
Synthetic notes inflate the NLP result	The limitation is stated openly and the NLP model is not used for deployment
Results change slightly between neural-network training runs	Report the figures of the saved model and state the run-to-run variation

The risks that matter most for a first version are class imbalance, bias and privacy. Each one is either handled in the design or stated openly in this document.

5. Tools and Techniques (initial assessment)

Table 3 gives the initial assessment of the tools and techniques, with the purpose of each and the reason it fits the problem.

Table 3: Tools and techniques

Tool or technique	Purpose	Why it fits
Python 3, Pandas, NumPy	Load, merge and engineer the data	Standard, free and well supported
Scikit-learn	Random Forest, train/test split, metrics	Reliable baseline for tabular data
imbalanced-learn (SMOTE)	Balance the classes in the training data	Improves precision on the at-risk class
TensorFlow/Keras	Neural network for the deep learning comparison	Allows a fair comparison with the Random Forest

Tool or technique	Purpose	Why it fits
TextBlob	Sentiment scores from teacher notes	Simple and free NLP library
FastAPI and Uvicorn	Risk API and chatbot with automatic /docs page	Quick to build and easy to test
GitHub and GitHub Projects	Version control, pull requests and task board	Required by the project brief

Every tool is free and open source, which keeps the solution affordable for institutions with small budgets.

6. Machine Learning Approach

The problem is supervised binary classification: predict whether a student is At-Risk (final grade G3 below 10 out of 20, the pass mark of the Portuguese system where the data was collected) from earlier academic and behavioural indicators. G3 is used only to create the label and never as an input, so the model cannot see the answer. Related studies have applied tree-based classifiers and neural networks to student performance data (Cortez and Silva, 2008; Amrieh, Hamtini and Aljarah, 2015; Hussain et al., 2018). Hussain et al. (2018) compared four classifiers on college data and found Random Forest the strongest, which supports our choice of a Random Forest baseline. We chose algorithms that suit small tabular data and that can be compared fairly:

- Random Forest baseline (Breiman, 2001; 100 trees, class_weight='balanced'): robust, needs little tuning and gives feature importance for explanations.
- Random Forest + SMOTE: balances the classes by creating synthetic at-risk examples in the training data only, using the SMOTE method (Chawla et al., 2002).
- Random Forest + SMOTE + NLP features: tests whether teacher-note sentiment adds information.
- Keras neural network: tests whether deep learning improves on the Random Forest.

All models use the same 80/20 stratified split with random_state=42: 835 training students and 209 test students (46 at-risk).

7. Data

7.1 Source

The Student Performance dataset created by Cortez and Silva (2008) and downloaded from Kaggle (see Section 17): 33 attributes covering demographics, family background, study habits and the grades G1, G2 and G3, for 395 Mathematics students and 649 Portuguese students (1,044 records combined).

7.2 Forms of data and examples

The solution uses three forms of data: structured (tabular) data, text data and time-series data. Table 4 shows structured data, Table 5 shows text data with its sentiment scores, and Table 6 in Section 8 shows the time-series data.

Table 4: Structured data: raw Mathematics records (first 6 columns, first 3 rows)

school	sex	age	address	famsize	Pstatus
GP	F	18	U	GT3	A
GP	F	17	U	GT3	T
GP	F	15	U	LE3	T

These columns describe the school, sex, age and family background of each student. The deployed model uses age, address, famsize and Pstatus from this group (Section 7.3).

Table 5: Text data: synthetic teacher notes with their TextBlob sentiment scores (first 3 rows)

Student ID	Teacher note	Polarity	Subjectivity	Negative flag
1	Attends regularly and completes the required work.	0.0	0.076923	0
2	Attends regularly and completes the required work.	0.0	0.076923	0
3	Struggling badly with the basics, needs urgent support.	-0.7	0.666667	1

The three score columns are stored as sentiment_polarity, sentiment_subjectivity and sentiment_negative_flag (Section 9). Student 3's note is negative (-0.7) and is flagged, while the neutral notes of students 1 and 2 score 0.0.

7.3 Features used by the deployed model

The deployed model uses 11 features: age, studytime, failures, absences, g1_to_g2_change, avg_g1_g2, address, famsize, Pstatus, schoolsup and famsup.

8. Time Series Analysis

The dataset has no weekly observations, so the three sequential grading periods G1, G2 and G3 are treated as discrete time steps. Two features capture the trend in each student's performance: g1_to_g2_change (G2 minus G1) shows whether marks are rising or falling, and avg_g1_g2 (the mean of G1 and G2) shows the overall level. G3 is kept out of the inputs and used only for the label (a student is At-Risk when G3 is below 10). Table 6 shows the grades at the three time steps together with the two features and the label for the first five students.

Table 6: Time-series data and the features built from it (first 5 students)

Student ID	G1	G2	G3	G1 to G2 change	Average of G1 and G2	At-risk label
1	5	6	6	1	5.5	1
2	5	5	6	0	5.0	1
3	7	8	10	1	7.5	0
4	15	14	15	-1	14.5	0
5	6	10	10	4	8.0	0

In the table, the two feature columns are stored as g1_to_g2_change and avg_g1_g2, and the label column as label_at_risk. Student 5 improved by 4 marks between G1 and G2 and passed with a G3 of 10, while students 1 and 2 stayed low and were labelled at-risk. The example for student 73 in Section 13 shows the value of these features: the reason for the High flag names an average of 7, two previous failures and a grade change of -2 between G1 and G2. In a live deployment the same method would be applied to weekly data, using weekly changes and moving averages.

9. Natural Language Processing

The dataset has no free-text field, so teacher comments were generated from each student's absences, previous failures and grade trajectory (G1 and G2). The generator never sees G3 or the at-risk label. In a live system these comments would come from real learning-management-system entries. Each note is scored with TextBlob, which adds three columns: sentiment_polarity (from -1 negative to +1 positive), sentiment_subjectivity and sentiment_negative_flag. Table 5 in Section 7.2 shows examples of the notes and their scores.

Result: adding the sentiment features made every metric slightly worse (F1 0.78 to 0.76). This is the expected, honest outcome: the notes were built from features already in the table, so the sentiment adds noise instead of new information. NLP is relevant to the theme because real teacher comments often contain early warning signs that numbers miss. Speech recognition and speech synthesis are not used in this project.

10. Deep Learning

src/train_deep_learning.py trains a Keras neural network on the same features and the same split as the Random Forest. The settings are:

- Pre-processing: StandardScaler fitted on the training set only.
- Architecture: Dense(32, ReLU) → Dropout(0.3) → Dense(16, ReLU) → Dropout(0.2) → Dense(1, sigmoid).
- Parameters: 929 trainable (384 + 528 + 17).
- Training: Adam optimiser, binary cross-entropy, 50 epochs, batch size 16, 20% validation split.
- Reproducibility: NumPy and TensorFlow seeds set to 42.
- Output: models/deep_model.keras.

Dropout layers reduce over-fitting on a small dataset, and the sigmoid output gives a probability that is turned into a class at a 0.5 threshold. Even with fixed seeds, the figures changed slightly between training runs on our computers (precision 0.73 in one run and 0.76 in the next), so the figures reported in this document come from the final saved model.

Other deep learning techniques considered. A recurrent network (LSTM or GRU) is the natural choice when each student has weekly records, because it reads a sequence and remembers earlier weeks. We did not use one because the public dataset holds only three grading periods per student, which is too short for a sequence model to add value over the two trend features in Section 8. Deeper networks, convolutional networks and transformers were also considered and rejected: with 835 training students a larger network would over-fit, and the inputs are a small table and not images or long text. In a live deployment with weekly attendance and marks, an LSTM would be the first deep learning extension to test.

11. Model Evaluation and Solution Techniques

11.1 How the model is evaluated

Every model is tested on the same 209 unseen students. Because only 22% of students are at-risk, accuracy alone would be misleading, so we report precision, recall and F1 for the At-Risk class, together with a confusion matrix. Precision is the share of flagged students who are truly at-risk; recall is the share of at-risk students that the model catches. Missing an at-risk student costs more than a false alarm, so recall is the priority metric.

11.2 Results (At-Risk class, 209-student test set)

Table 7: Results for the four models

Model	Precision	Recall	F1	Accuracy
Random Forest baseline (class_weight='balanced')	0.64	0.89	0.75	0.87
Random Forest + SMOTE (no NLP)	0.71	0.87	0.78	0.89
Random Forest + SMOTE (with NLP)	0.70	0.85	0.76	0.89
Deep Learning (Keras)	0.76	0.83	0.79	0.90

All four models reach an accuracy of 0.87 to 0.90. However, accuracy alone is misleading when only 22% of students are at-risk; therefore, the models are compared

on precision and recall in Section 11.5. Table 8 shows the confusion matrix of each model. TN is the number of students who were not at-risk and were correctly cleared, FP is the number of false alarms, FN is the number of at-risk students who were missed, and TP is the number of at-risk students who were caught.

Table 8: Confusion matrix counts on the 209-student test set

Model	Cleared (TN)	False alarms (FP)	Missed (FN)	Caught (TP)
Random Forest baseline	140	23	5	41
Random Forest + SMOTE (no NLP)	147	16	6	40
Random Forest + SMOTE (with NLP)	146	17	7	39
Deep Learning (Keras)	151	12	8	38

The baseline misses the fewest at-risk students (5) but raises more false alarms (23). The Keras model raises the fewest false alarms (12) but misses the most at-risk students (8).

11.3 Techniques used to improve accuracy

- Class weighting and SMOTE to deal with the small number of at-risk students. SMOTE raised precision from 0.64 to 0.71 while recall moved only from 0.89 to 0.87.
- Time-series features (g1_to_g2_change and avg_g1_g2) that describe the trend in marks.
- Dropout and feature scaling in the neural network to reduce over-fitting.
- Feature experiments: the NLP features were tested and rejected because they did not help.
- Risk levels: the probability is converted into High (0.66 or more), Medium (0.33 to 0.65) and Low (below 0.33), so staff can choose how many students to follow up.

11.4 Risk levels on the test set (deployed model)

Table 9: Precision and recall of the API risk levels (deployed model)

Flag level	Students flagged	Truly at-risk	Precision	Recall
High (probability 0.66 or more)	46	35	0.76	0.76
Medium or High (probability 0.33 or more)	71	43	0.61	0.93

The High level meets the 70% precision target. Widening the net to Medium and High catches 93% of at-risk students but raises more false alarms, which is the trade-off staff can choose.

11.5 Model chosen for deployment

The team deployed Random Forest + SMOTE (no NLP). It has the highest precision of the Random Forest variants (0.71), it keeps strong recall (0.87), and it is the only Random Forest variant that meets the 70% precision target (the NLP variant scores 39 out of 56 flagged, which is 0.696 and rounds to 0.70 in Table 7, but is just below 70%). The Keras model has slightly higher precision (0.76) and F1 (0.79) but lower recall (0.83), and the test set has only 46 at-risk students, so one student changes recall by about two percentage points. Because the team treats recall as the priority, it kept the Random Forest.

12. Other Feature: Chatbot (/chat endpoint)

The chatbot gives academic staff plain-English intervention tips without requiring them to understand the model. It is rule-based, which keeps it predictable and safe for a first version. A request has three fields:

- message (text, required): the staff member’s question or topic.
- student_id (text, optional): used only to label the reply.
- features (object, optional): the same 11 features as /predict; when supplied they trigger a live risk score.

The chatbot behaves as follows:

- With features: the endpoint calls the model, returns the risk level and probability, and picks tips from the top contributing factors (absences, failures, grade change, average grade, study time).
- Without features: the endpoint matches keywords in the message to four topic banks (attendance, grades, previous failures, study habits). Asking about “risk level” returns the tier definitions.
- Every reply ends with the disclaimer: “These are suggestions for staff. Confirm with the student before acting: the risk score is a prompt to talk, not a verdict.”

13. Practical Solution

The practical solution is a working FastAPI service (src/api_server.py) that loads the deployed model and answers three requests. Start it with `python -m uvicorn src.api_server:app --reload`, and open http://127.0.0.1:8000/docs for the interactive page.

- GET health: returns the loaded model name and the number of features. Real answer: {"status": "ok", "model": "rf_smote_no_nlp.joblib", "n_features": 11}.
- POST predict: returns the risk probability, the risk level and a machine-generated reason for one student.
- POST /chat: returns intervention tips, with a live risk score when features are supplied.

13.1 Real examples from the running API

Predict Endpoint Examples (/predict)

Below are real examples of how university staff can use the /predict endpoint to assess student risk. By submitting the student's ID and feature set, the API returns a structured risk assessment including whether the student is at risk, the exact probability, risk level, and a human-readable explanation of the main contributing factors.

Note on the examples. Student 73 is a real row of the dataset, so its request contains that student's exact values. Example Student A and Example Student B are illustrative: the team typed their feature values by hand to show a Medium and a Low result, so the values are not rows of the dataset, and the identifiers "1" and "6" are only labels.

The answers shown are the real, unedited responses of the running API to these requests. The features address, famsize and Pstatus are sent as 0/1 codes.

1. High Risk (Student 73)

Request Payload (/predict):

JSON

```
{
  "student_id": "73",
  "features": {
    "age": 15.0,
    "studytime": 2.0,
    "failures": 2.0,
    "absences": 2.0,
    "g1_to_g2_change": -2.0,
    "avg_g1_g2": 7.0,
    "address": 0.0,
    "famsize": 0.0,
    "Pstatus": 1.0,
    "schoolsup": 1.0,
    "famsup": 1.0
  }
}
```

API Response:

JSON

```
{
  "student_id": "73",
  "at_risk": true,
  "risk_probability": 1.0,
  "risk_level": "High",
  "reason": "Main factors: average of G1 and G2 grades = 7 (below average); prior failures = 2 (above average); change in grade from G1 to G2 = -2 (below average)"
}
```

2. Medium Risk (Example Student A)

Request Payload (/predict):

JSON

```
{
  "student_id": "1",
  "features": {
    "age": 18,
    "studytime": 2,
    "failures": 0,
    "absences": 6,
    "g1_to_g2_change": 1,
    "avg_g1_g2": 5.5,
    "address": 0,
    "famsize": 1,
    "Pstatus": 1,
    "schoolsup": 1,
    "famsup": 0
  }
}
```

API Response:

JSON

```
{
  "student_id": "1",
  "at_risk": true,
  "risk_probability": 0.63,
  "risk_level": "Medium",
  "reason": "Main factors: average of G1 and G2 grades = 5.5 (below average); famsize = 1 (above average); schoolsup = 1 (above average)"
}
```

3. Low Risk (Example Student B)

Request Payload (/predict):

JSON

```
{
  "student_id": "6",
  "features": {
    "age": 16,
    "studytime": 2,
    "failures": 0,
    "absences": 10,
    "g1_to_g2_change": 0,
    "avg_g1_g2": 15,
    "address": 0,
    "famsize": 0,
    "Pstatus": 0,
    "schoolsup": 0,
    "famsup": 1
  }
}
```

API Response:

JSON

```
{
  "student_id": "6",
  "at_risk": false,
  "risk_probability": 0.07,
  "risk_level": "Low",
  "reason": null
}
```

No reason is given for a Low result, because there is no flag to explain.

The reason lists the features that differ most from the training average, weighted by how important each feature is to the model. It does not show whether a factor raises or lowers the risk, so staff should read it as a starting point for a conversation.

Chat Assistant Endpoint Examples (/chat)

Below are real examples of how university staff can interact with the /chat endpoint. By sending a custom message along with the student's ID and feature set, the API returns a risk-aware response complete with actionable next steps and tailored intervention tips.
The /chat examples reuse the same inputs as the /predict examples above.

1. Medium Risk (Example Student A)

Request Payload:

JSON

```
{
  "message": "What steps can I take to help this student improve?",
  "student_id": "1",
  "features": {
    "age": 18,
    "studytime": 2,
    "failures": 0,
    "absences": 6,
    "g1_to_g2_change": 1,
    "avg_g1_g2": 5.5,
    "address": 0,
    "famsize": 1,
    "Pstatus": 1,
    "schoolsup": 1,
    "famsup": 0
  }
}
```

API Response:

JSON


```
{
  "reply": "Student 1: Medium risk (probability 0.63).\nSuggested next step: check in with the student within two weeks and watch the next assessment.\nTips:\n1. Arrange extra support for the weaker topics (tutoring, past papers, office hours) and set one small target for the next assessment.\nThese are suggestions for staff. Confirm with the student before acting: the risk score is a prompt to talk, not a verdict.",
  "risk_level": "Medium",
```

```
  "risk_probability": 0.63,
  "tips": [
 "Arrange extra support for the weaker topics (tutoring, past papers, office hours) and set one small target for the next assessment."
  ]
}
```


2. Low Risk (Example Student B)

Request Payload:

JSON


```
  {
    "message": "Does this student require any immediate academic intervention?",
    "student_id": "6",
    "features": {
      "age": 16,
      "studytime": 2,
      "failures": 0,
      "absences": 10,
      "g1_to_g2_change": 0,
      "avg_g1_g2": 15,
      "address": 0,
      "famsize": 0,
      "Pstatus": 0,
      "schoolsup": 0,
      "famsup": 1
    }
  }
```


API Response:

JSON


```
{
"reply": "Student 6: Low risk (probability 0.07).\nNo intervention is needed now; keep monitoring.\nThese are suggestions for staff. Confirm with the student before acting: the risk score is a prompt to talk, not a verdict.",
  "risk_level": "Low",
  "risk_probability": 0.07,
  "tips": []
}
```

3. High Risk (Student 73)

Request Payload:

JSON

```
{
  "message": "What intervention steps should be taken for this high-risk student?",
  "student_id": "73",
  "features": {
    "age": 15,
    "studytime": 2,
    "failures": 2,
    "absences": 2,
    "g1_to_g2_change": -2,
    "avg_g1_g2": 7,
    "address": 0,
    "famsize": 0,
    "Pstatus": 1,
    "schoolsup": 1,
    "famsup": 1
  }
}
```

API Response:

```
JSON
{
  "reply": "Student 73: High risk (probability 1.00).\nSuggested next step: contact the student this week, and record the intervention so its effect can be measured.\nTips:\n1. Arrange extra support for the weaker topics (tutoring, past papers, office hours) and set one small target for the next assessment.\n2. Review the earlier failed subject(s) with the student, find the topics that caused the problem, and offer catch-up tutoring or a peer study partner.\n3. Grades are falling between periods: ask what changed and review the most recent assessments to see where marks dropped.\nThese are suggestions for staff. Confirm with the student before acting: the risk score is a prompt to talk, not a verdict.",
  "risk_level": "High",
  "risk_probability": 1,
  "tips": [
    "Arrange extra support for the weaker topics (tutoring, past papers, office hours) and set one small target for the next assessment.",
    "Review the earlier failed subject(s) with the student, find the topics that caused the problem, and offer catch-up tutoring or a peer study partner.",
    "Grades are falling between periods: ask what changed and review the most recent assessments to see where marks dropped."
  ]
}
```

These examples show the three links of the solution working together: data features, a model prediction and a human-readable explanation that staff can act on, through both the /predict and /chat endpoints.

14. Use of AI and Ethical Disclosure

In alignment with academic guidelines, generative AI and technical assistant tools (specifically Claude and IBM Bob) were utilized solely as technical advisory,

debugging, and mentoring partners during the development of the Ubuntu-Connect project.

- Role of AI: These tools provided troubleshooting support for Python syntax, guidance on structuring FastAPI endpoints (/predict and /chat), assistance in clarifying technical documentation and markdown structures, and help with drafting and formatting this document from the team's own results.
- Team Responsibility: All core architectural decisions, dataset merges, pipeline executions, model training, code implementations, validation tests, and final reviews were entirely conducted, verified, and executed by the student project team members. No AI tool was used to bypass academic rigor or fabricate project results.

15. Project Management and Team

All work is stored on GitHub (https://github.com/SiyabongaBlessingNzotho/Ubuntu-connect), and tasks were tracked with a GitHub Project board. Work was merged through pull requests. The nine members and the stage each one built are listed in the Declaration at the start of this document.

16. Conclusion

Ubuntu-Connect shows that data an institution already holds can warn staff about at-risk students early and explain why. The deployed Random Forest + SMOTE model flags High-risk students with 0.76 precision, above the 70% target, and the Medium and High levels together catch 93% of at-risk students.

The Keras comparison, the NLP experiment and the chatbot show how each technique in the module contributes to the solution. The main limits are the small test set, the synthetic teacher notes and the absence of real intervention data, so the next steps are a pilot with consented data, a fairness review of the demographic features, a dashboard for staff, and measuring whether intervention lowers the fail rate.

17. References

Amrieh, E. A., Hamtini, T. and Aljarah, I. (2015) Preprocessing and analyzing educational data set using X-API for improving student's performance. 2015 IEEE

Jordan Conference on Applied Electrical Engineering and Computing Technologies (AEECT).

Breiman, L. (2001) Random forests. Machine Learning, 45(1), pp. 5-32.

Chawla, N. V., Bowyer, K. W., Hall, L. O. and Kegelmeyer, W. P. (2002) SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, pp. 321-357.

Cortez, P. and Silva, A. (2008) Using data mining to predict secondary school student performance. Proceedings of the 5th Annual Future Business Technology Conference (FUBUTEC), Porto, Portugal.

Data-Science Sean (2019) Student Performance Data Set [Data set]. Kaggle. Available at: https://www.kaggle.com/datasets/larsen0966/student-performance-data-set (Accessed: 28 October 2026).

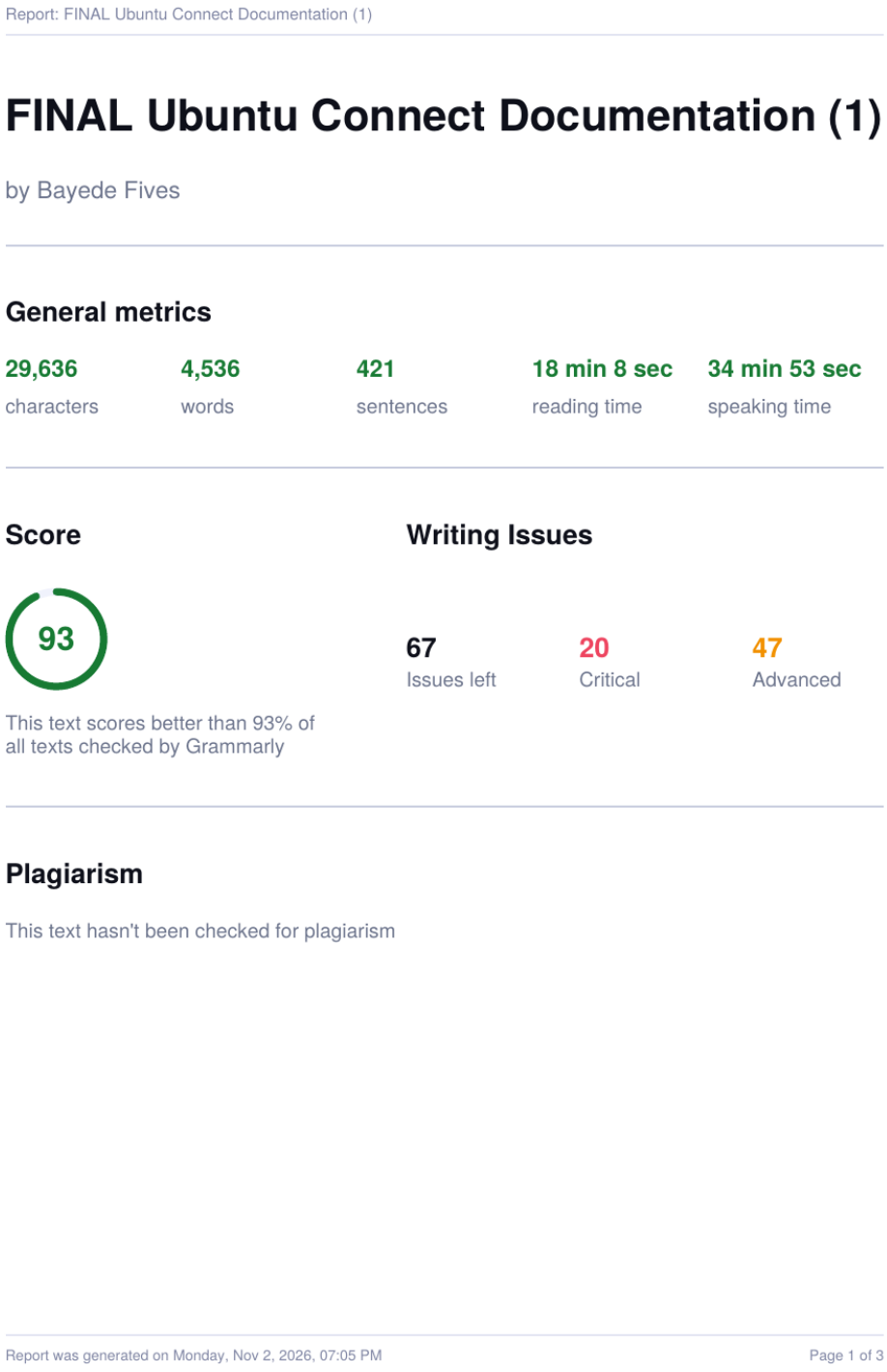
Hussain, S., Dahan, N. A., Ba-Alwi, F. M. and Ribata, N. (2018) Educational data mining and analysis of students' academic performance using WEKA. Indonesian Journal of Electrical Engineering and Computer Science, 9(2), pp. 447-459.

Pedregosa, F. et al. (2011) Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, pp. 2825-2830.

Open-source software documentation: imbalanced-learn, TensorFlow/Keras, TextBlob, FastAPI and Pandas.

Appendix A: Grammarly Report

The following pages contain the Grammarly report for this documentation, generated on 2 November 2026.



Report: FINAL Ubuntu Connect Documentation (1)

FINAL Ubuntu Connect Documentation (1)

by Bayede Fives

General metrics

29,636	4,536	421	18 min 8 sec	34 min 53 sec
characters	words	sentences	reading time	speaking time

Score

93

This text scores better than 93% of all texts checked by Grammarly

Writing Issues

67	20	47
Issues left	Critical	Advanced

Plagiarism

This text hasn't been checked for plagiarism

Report was generated on Monday, Nov 2, 2026, 07:05 PM

Page 1 of 3

Writing Issues

1	**Clarity**	
1	Paragraph can be improved	
46	**Pro suggestions**	
2	Hard-to-read text	
43	Paragraph can be improved	
1	Incomplete sentences	
20	**Correctness**	
7	Incorrect punctuation	
1	Incorrect verb forms	
2	Comma misuse within clauses	
3	Misspelled words	
1	Improper formatting	
1	Incorrect phrasing	
3	Mixed dialects of English	
1	Confused words	
1	Incorrect noun number	

Unique Words

Measures vocabulary diversity by calculating the percentage of words used only once in your document

20%

unique words

Rare Words

Measures depth of vocabulary by identifying words that are not among the 5,000 most common English words.

41%

rare words

Word Length

Measures average word length

4.8

characters per word

Sentence Length

Measures average sentence length

10.8

words per sentence